Lesson 9 Teacher's Script

Core Mechanics III: On the Scoreboard

VIDEO GAME CREATION

[0:00-0:05] Welcome Back

"Welcome back! Last week your game got real stakes - lives, and a lose condition. Quick show of hands - who tested both directions, shooting AND escaping, to make sure the two counters stayed separate?" [Pause, call on 1-2 volunteers]

"Today, your game finally gets a real scoreboard - not just a message at the end, but score and lives visible the whole time you're playing. Today's demo is also back in GameMaker - same engine as Lesson 5, but this time in its typed GML mode instead of GML Visual, so keep an eye out for how much of what you already learned there still applies."

[0:05-0:15] The Payoff of Deferring UI

"Way back in Lesson 6, I told you something on purpose: 'a plain line of text is completely legitimate today - the real UI system comes later, once there's a score and lives to actually display.' That was a promise. Today we keep it."

"Here's the interesting part: for some of you, keeping that promise is almost free. Your engine's win/lose message tool was ALREADY the real tool - it just needs to run all the time instead of once. For a couple of you, today introduces something genuinely new."

[Discussion question] "Why do you think we waited until now to build a real UI, instead of doing it back in Lesson 6 alongside the win message?" (Draw out: there was nothing worth displaying continuously yet - a UI showing nothing changing isn't useful; build the data first, display it once it means something.)

[Timing checkpoint: wrap by 0:15]

[0:15-0:35] Score: Reusing the Hit-Detection Path

"First, the easy part: score. Every engine's hit-detection code already runs exactly once per confirmed kill - the same place enemies_remaining already changes. Score just joins it."

[Demo cue: switch to GameMaker, open a project with working lives from Lesson 8]

"Watch: right in obj_bullet's Collision event with obj_enemy - the exact same event where enemies_remaining already goes down - I add one line: obj_gamemanager.__dnd_score += 10. That's the whole scoring system (__dnd_score is the same variable GameMaker's drag-and-drop Score blocks use, which is why it has that odd name). The hard part isn't the math, it's showing it." [Add the __dnd_score += 10 line, show it's not visible yet]

"To actually SHOW score and lives, watch where I put the display code: obj_gamemanager's Draw GUI event - the exact same event that's already been drawing 'You Win!' and 'Game Over' since Lesson 6. I add two plain draw_text() calls there, with no if-check around them at all, so they just run every single frame no matter what." [Add the two unconditional draw_text() calls, run the game, shoot an enemy and let one escape to show both numbers update live]

[Timing checkpoint: wrap by 0:35]

[0:35-0:50] Two Ways to Stay in Sync: Push vs. Poll

"What I just showed you in GameMaker is called POLL - instead of remembering to push an update every time something changes, you just check the data every single frame, no matter what. GameMaker's Draw GUI event and TIC-80's print() calls have quietly been doing this since Lesson 6 - today, Unity's HUD script does it on purpose too."

"But there's a second, equally valid way, and most of the rest of you will use it today: PUSH. Every time the data actually changes, you push an update to the display, instead of checking it constantly. Godot, GDevelop, Unreal, and 001 Game Creator's real-time Field binding all work this way."

[Demo cue, optional: pull up Godot briefly] "Neither approach is 'more correct.' Poll means you never forget to update, but you're technically doing a little more work than necessary, every frame, forever. Push means less wasted work, but you have to remember to call the update everywhere something changes. For two small numbers like these, the cost either way is nothing."

[Discussion question] "Can you think of something in a game where checking every single frame would start to feel wasteful, if there were hundreds of things to check instead of two?" (Open-ended; validate any reasonable guess - this previews ideas about performance they'll meet again in later, more advanced work.)

[Timing checkpoint: wrap by 0:50]

[0:50-0:55] Today's Exercise & Handouts

"For lab today, open your own project from last week - not a new one, and no new assets needed. You're adding score tracking and a real, always-visible HUD."

[Hand out engine-specific handouts]

"Unity, Unreal, and 001 Game Creator folks - your setup today is more involved than usual, genuinely new UI tools. That's expected, not a sign you're behind. Everyone else - your engine's already had the right tool since Lesson 6, today's just about using it continuously."

"Questions before we start lab? ... Let's get your scoreboard up."

Lab Time Notes

- Unity, Unreal, and 001 Game Creator clusters need the most attention this week - budget extra circulation time there, and expect setup-related questions before logic questions.
- 001 Game Creator: the single most likely mistake is forgetting to tick "Update Real-time" on a Field - it looks like a mysterious bug (frozen numbers) rather than an obviously missing setting. Check this first if a student's HUD isn't updating.

- Unity: NullReferenceException on Play almost always means a Text object wasn't actually dragged into the HUD script's Inspector fields - check this first.
- A score readout stuck at 0 is almost always a missing or misplaced increment, not a display bug - have students confirm the underlying variable is actually changing before troubleshooting the display.
- Students who finish Level 2 early should be pointed at the Level 3 ideas on their handout (HUD styling is a satisfying, visible one), not given new unrelated work.